Recover the Design

Date: 2026-10-23

Abstract

We recover a fence layout from the degrees of the connectors on a grid. We treat each possible straight fence between two neighbouring connectors as an edge, and the task becomes choosing a set of edges. First we use degree sums to reject inputs that can never work. Then we turn the rules into Boolean clauses and solve them with a small SAT solver that uses backtracking and learns from conflicts. The program passes all our tests, and generated 50 by 50 inputs finish in less than 0.2 seconds.

Contents

1 Introduction **3**

2 Data Structure / Algorithm Specification **3**

2.1 Candidate Edges 3

2.2 Quick Checks 3

2.3 Clauses 4

2.4 Search 4

2.5 Correctness 5

3 Testing Results **5**

3.1 Method 5

3.2 Correctness Tests 6

3.3 Timing Results 7

4 Analysis and Comments **7**

4.1 Time Complexity 7

4.2 Space Complexity 8

4.3 Discussion of Test Results 8

4.4 Limitations and Improvements 9

5 Conclusion **9**

6 References **9**

7 Source Code Appendix **9**

1 Introduction

In the problem "Recover the Design", we are given an $n \times m$ grid ($1 < n, m \leq 50$). A cell with 0 is empty, and a cell with k (1 to 4) is a connector that must have fences in exactly k of its four directions. Fences are horizontal or vertical. They can only turn or cross at connectors, not in empty cells. If a fence goes straight through a connector, it uses two directions of that connector.

For each connector, in row order and from left to right, we print `row col up down left right`, where each direction is 0 or 1. Numbers are separated by one space. If there is no layout we print `No Solution`. Any valid layout is accepted.

The difficulty is that the choices affect each other. A 50 by 50 grid can have almost 5,000 possible fences, so we can not try all 2^{5000} subsets. Choosing a fence changes the degree of both of its ends, and a long fence can block another fence far away. So we need a clear model, some quick checks, and a search that can go back when a choice turns out wrong.

2 Data Structure / Algorithm Specification

2.1 Candidate Edges

Each connector is a vertex. If two connectors are in the same row or column with only empty cells between them, we connect them with a candidate edge. So each connector has at most four edges, one for each direction. A long fence through several connectors is just several edges in a line, so we do not need edges that jump over a connector.

Let $x_e = 1$ if edge e is used and $x_e = 0$ otherwise. The rules become:

1. For each connector v with degree k_v: $\sum_{e \ni v} x_e = k_v$.
2. If a horizontal edge e and a vertical edge f pass the same empty cell: $x_e + x_f \leq 1$.

We find the edges by scanning each row and each column once. While adding an edge, we mark the empty cells it passes in a horizontal grid or a vertical grid. A cell marked in both grids gives a crossing pair. For example, if the center of a 3 by 3 grid is empty and the four cells next to it are connectors, the middle-row edge and the middle-column edge cross at the center, so they can not both be used.

2.2 Quick Checks

Every used edge adds 1 to the degree of both of its ends. So in each connected part of the graph, the sum of the degrees must be even. We also try to color each part with two colors using BFS. If this works (the part is bipartite), each edge joins two different colors, so both colors must have the same degree sum. If either check fails, we print `No Solution` at once. For example, a grid full of 1s with an odd number of cells is rejected here.

We color the graph itself instead of using the chessboard color $(r + c) \bmod 2$, because a long edge such as $(1, 1)$–$(1, 3)$ joins two cells of the same chessboard color.

2.3 Clauses

After the quick checks, we still need to search for the edges. Our first version tried the edges one by one with plain backtracking and checked the two rules directly in the code. It was slow on some large inputs, because it often failed for the same reason again and again in different branches, and it had no way to remember why a choice failed. So we decided to write every rule in the same simple form, a clause. A clause is a list of conditions joined by "or", and every clause must be true. Each edge is one Boolean variable (used or not used). With all rules in this form, one search procedure can handle both rules, and when the search fails we can save the reason as one more clause of the same form.

The crossing rule is easy: for a crossing pair e, f we add $(\neg x_e \vee \neg x_f)$, which means "not both".

The degree rule "exactly k of its d edges are used" is not an "or" by itself, so we split it into two parts. "At most k" means any $k + 1$ of the edges can not all be used, so at least one of them is unused. "At least k" means any $d - k + 1$ of the edges can not all be unused, so at least one of them is used. For example, with $d = 3$ edges a, b, c and $k = 2$, we get $(\neg a \vee \neg b \vee \neg c)$, $(a \vee b)$, $(a \vee c)$ and $(b \vee c)$. Listing all subsets sounds slow, but $d \leq 4$, so each connector gives at most a few short clauses. If $k > d$, we print `No Solution` at once.

2.4 Search

We solve the clauses with a small SAT solver that we wrote ourselves, following the CDCL method in [1]. The solver repeats three steps:

1. **Propagation.** If all literals of a clause are false except one, that one must be true, so we set it. To do this quickly, each clause only keeps track of two of its literals, and we check a clause only when one of these two becomes false.
2. **Guess.** If nothing is forced, we pick an edge that has no value yet and guess a value. If all edges have values, we have found a layout.
3. **Conflict.** If some clause becomes false, the last guesses were wrong. We follow the reasons of the forced values back to find which guesses caused the conflict, and save this as a new clause. Then we go back to an earlier guess and continue. Because the new clause is kept, the same wrong combination is never tried again.

If a conflict happens before any guess, there is no solution. We also start over from the beginning after some number of conflicts (100 at first, growing by 1.5 times), keeping the learned clauses. Edges that often appear in conflicts are guessed first.

2.5 Correctness

Every legal layout can be cut at connectors into candidate edges, and these edges satisfy all clauses. On the other hand, any set of edges that satisfies all clauses has the right degree at every connector and no crossing in an empty cell, and the fences are straight because each edge is straight. So the clauses describe exactly the legal layouts.

The quick checks only reject inputs whose degree sums can not match any edge set. Each learned clause follows from the old clauses, so it never removes a legal layout [1]. Going back removes guesses but never clauses. So if the solver finds values for all edges, the printed layout is legal, and if it gets a conflict without any guess, `No Solution` is correct.

3 Testing Results

3.1 Method

All tests were run on Linux (Intel Xeon 2.10 GHz, GCC 13.3.0, Python 3.13). We compile with `-O2`, and also with AddressSanitizer and UndefinedBehaviorSanitizer to find memory errors.

We wrote a checker in Python. It follows every printed direction cell by cell and checks the degrees, that both ends of each fence agree, the output format, and that no fences cross in an empty cell. For small grids we also try all subsets of edges by brute force to know whether a solution exists.

3.2 Correctness Tests

Test group (count)	Result	Purpose
Sample input (1)	PASS	The example from the problem gives a legal layout.
All 2 by 2 grids (625)	PASS	Smallest size; every input is compared with brute force.
Thin grids (152)	PASS	$n = 2$ or $m = 2$; random small ones checked by brute force, plus 2 by 50 and 50 by 2.
Random 3 by 3 grids (200)	PASS	Many inputs with no solution; compared with brute force.
Hand-made cases (4)	PASS	Crossing in an empty cell is refused; crossing at a degree 4 connector is allowed; a fence goes straight through a connector; a square with two solutions.
Several solutions (50)	PASS	3 by 4 grids with at least two layouts; any legal one is accepted.
Planted 4 by 4 grids (100)	PASS	Grids made from random fences, so they always have a solution.
Special 50 by 50 grids (5)	PASS	Largest size: empty, all 1s, full grid, long fences, all 4s (no solution).
Degree sum cases (9)	PASS	Odd degree sum or unequal color sums (no solution), and solvable grids that the quick checks must not reject.
Planted 20 by 20 and 50 by 50 (24)	PASS	Large grids with a solution; each must finish within 2 seconds.

All 1,170 cases pass in both builds, and the sanitizers report no error. We also tested the SAT solver alone on 2,000 small random formulas against full truth tables, and all of them pass. To check that the conflict and go-back part really runs, we added counters to a copy of the program: on a planted 50 by 50 grid it met 412 conflicts and went back 414 times before finding a layout.

3.3 Timing Results

Large inputs are made by our generator `code/gen.cpp`. It makes three kinds of grids:

1. **planted**: place random fences that do not cross and write down the degrees, so there is always a solution.
2. **broken**: a planted grid with 1 added to one connector and 1 taken from another, so the quick checks often can not reject it. It may or may not have a solution.
3. **random**: random degrees from 1 to 4. These almost never have a solution.

We test n by n grids with $n = 10, 20, 30, 40, 50$ and five seeds each. Every input is run once to warm up and then 10 times. The table shows the median time in milliseconds over the five inputs. The time includes about 2 ms for starting the program.

Input kind	10	20	30	40	50
planted 1.0	2.6	3.7	9.7	42.0	121.6
planted 0.5	2.5	2.7	3.8	6.1	8.8
broken 1.0	2.7	4.0	8.7	36.5	95.2
random 0.6	2.4	2.5	2.9	2.9	3.1

The number after each kind is the chance that a cell has a connector. The figure shows the same data on a log scale; the error bars go from the fastest to the slowest input.

4 Analysis and Comments

4.1 Time Complexity

Let V be the number of connectors and E the number of candidate edges. Since each connector has at most four edges, $E \le 2V \le 2nm$. Reading the grid, building the edges, finding crossings, the quick checks and making the clauses are all $O(nm)$, because each connector gives only a constant number of clauses.

For the search, let L be the total length of all clauses, including learned ones, and K the number of conflicts. Between two conflicts at most E edges get a value. Propagation in this round costs at most $O(EL)$, and choosing edges costs $O(E^2)$ because each choice scans all edges. So one round costs $O(E(E + L))$ and the whole search costs $O((K + 1)E(E + L))$. We can not bound K by a polynomial: in the worst case the search may need an exponential number of conflicts, so the worst-case time is exponential.

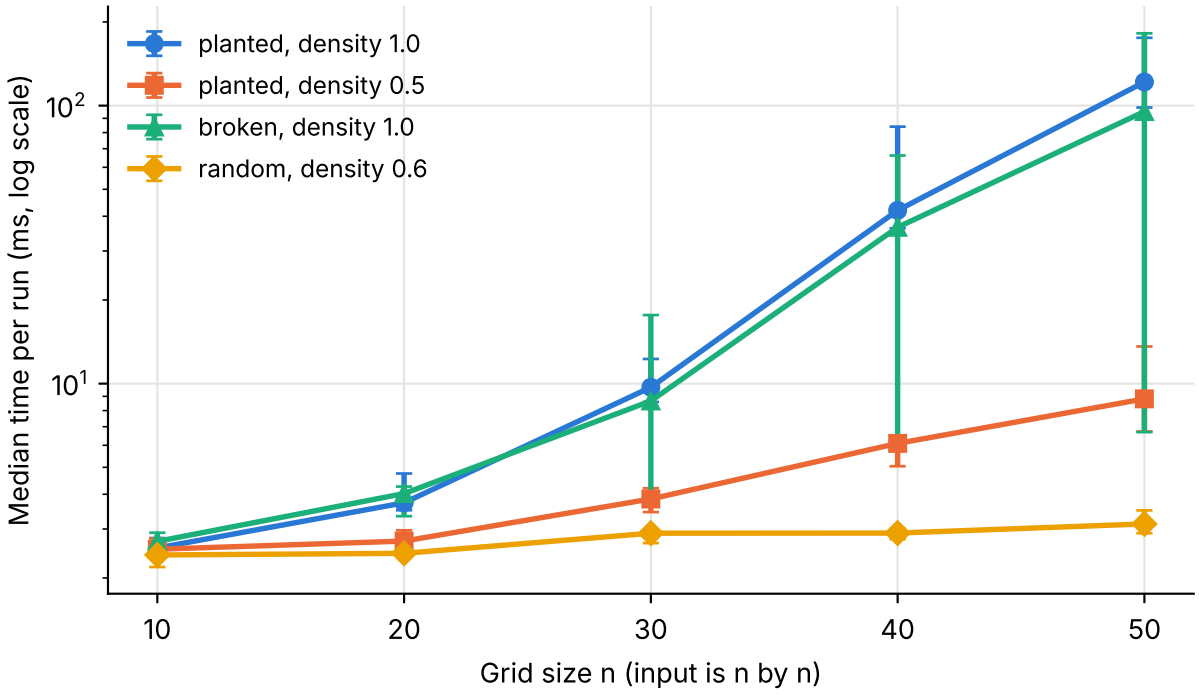


Figure 1: Median running time on generated n by n inputs

4.2 Space Complexity

The grid, the vertices and the edges take $O(nm)$ space. The clauses take $O(L)$ and the arrays of the solver take $O(E)$. The total space is $O(nm + L)$. Since learned clauses are never deleted, L can also become very large in the worst case.

4.3 Discussion of Test Results

Random grids take about 3 ms at every size, which is mostly program start-up, because the quick checks reject them before any search. Planted grids with density 0.5 stay below 10 ms, because many connectors have only one or two edges and propagation decides most of them.

Dense grids (density 1.0) are the slowest. Every connector has up to four choices, so the solver meets hundreds to more than a thousand conflicts. From 30 by 30 to 50 by 50 the grid is 2.8 times larger, but the time is about 12 times larger, because K and L also grow with the grid. Broken grids are like planted ones when they still have a solution; when they do not, some are rejected quickly and some need a real search. Still, every input we tried finished in less than 0.2 seconds.

4.4 Limitations and Improvements

1. The worst case is exponential, and our generated tests may miss the hardest inputs. We have not tested on the hidden test cases of the judge.
2. Learned clauses are never deleted, so memory may grow on hard inputs. Deleting old learned clauses that are no longer useful would help.
3. Choosing the next edge scans all edges in $O(E)$. Keeping the free edges in a binary heap ordered by score would make it $O(\log E)$.

5 Conclusion

We modeled the fences as edges between neighbouring connectors and wrote the degree and crossing rules as clauses. Simple degree sum checks reject many impossible inputs, and a SAT solver with backtracking and learning solves the rest. Our tests show the program is correct on all small inputs compared with brute force, and fast on 50 by 50 inputs, although the worst case is still exponential.

6 References

[1] N. Eén and N. Sörensson, "An Extensible SAT-solver," in *Theory and Applications of Satisfiability Testing*, LNCS, vol. 2919. Berlin, Heidelberg, Germany: Springer, 2004, pp. 502–518. doi:10.1007/978-3-540-24605-3_37.

7 Source Code Appendix

The source code is in the `code/` folder of the submission. `code/main.cpp` is the solver: class `Solver` reads the grid, builds the edges and clauses and prints the answer, and class `BooleanSolver` is the SAT solver. `code/gen.cpp` is the generator for the large tests. The `README.md` shows how to compile and run them.